Predicting Strikeout Rate

Ian Chang

2026-08-26

Introduction

The goal for this project was to predict each player's K% from the 2025 season. The data was collected from fangraphs and consisted of pitchers with at least 1 batter faced from the 2021-2025 season. The data contained only information about the pitchers total K%, total batters faced, age, and Stuff+. I wanted to incorporate more statistical data for each pitcher so I obtained data from Baseball Savant (https://baseballsavant.mlb.com/ (https://baseballsavant.mlb.com/)), and joined it with the fangraphs data. Throughout this R Markdown file, I will explain my data, methodology, model ramifications, and analyze how my model performs on the 2025 players. This file contains only my own work, and is formatted to be completely reproducible.

Setup & Data

```
library(tidyverse)
library(ggplot2)
library(ggrepel)
library(lightgbm)
library(xgboost)

# stats.csv attached as a downloadable file
stats <- read.csv('~/Downloads/stats.csv')
data <- read.csv('~/Downloads/k_2026.csv')
```

Data Description

For our model, I wanted to include pitcher statcast data such as `whiff%`, `called strikes`, and pitch arsenal data including fastball/breaking/offspeed pitch usage, speed, spin, and etc. I was able to download that data from Baseball Savant, which included 48 custom pitcher stat columns of players with a minimum of 1IP from 2021-2025. Luckily both the Savant and Fangraphs data used the same `Player ID` (which I verified), so I was able to join the data together, which resulted in a total of 50 columns describing each pitchers stats from that season.

Pitcher metrics included `called_strikes`, `swinging_strikes`, `zone_swing&miss%`, `out-zone_swing%`, `whiff%`, `Stuff+`, `fastball_usage`, `breaking_usage`, `offspeed_usage`, `fastball_avg_speed`, `breaking_avg_speed`, `offspeed_avg_speed`, `K%`, `TBF`, among others. Based on my intuition these were metrics that could most impact a players next year strikeout rate. And the idea was that I could use a players current season stats and difference in their stats from the previous season(s) to predict their next season `K%`.

Data Engineering

When joining the data, I immediately ran into data quality issues. There were many NaN's, different column names, and missing data due to some players not having played in the previous season. The following shows my work in removing NaN's, renaming columns, and engineering a couple features.

```
# Renaming data columns to match each other and provide better readability
data <- data %>%
  rename('K%' = 'K.',
         'Stuff+' = 'Stuff.')
stats <- stats %>%
  rename('last_name, first_name' = 'last_name..first_name') %>%
  select(-c('k_percent'))

# Identify NA's
data %>%
  summarise(across(everything(), ~ sum(is.na(.))))
```

```
##   Season Name PlayerId Team K% TBF Stuff+ Age
## 1      0    0        0    0  0   0      0   0
```

```
stats %>%
  summarise(across(everything(), ~ sum(is.na(.))))
```

```
##   last_name, first_name player_id year player_age p_formatted_ip bb_percent
## 1                     0         0    0          0              0          0
##   p_called_strike p_swinging_strike p_total_pitches p_total_strike
## 1               0                 0               0              0
##   p_total_swinging_strike avg_best_speed avg_hyper_speed z_swing_percent
## 1                       0              0               0               0
##   z_swing_miss_percent oz_swing_percent oz_swing_miss_percent out_zone_percent
## 1                    1                0                     6                0
##   pitch_count_offspeed pitch_count_fastball pitch_count_breaking
## 1                    0                    0                    0
##   in_zone_percent edge_percent whiff_percent swing_percent f_strike_percent
## 1               0            0             0             0                0
##   n_fastball_formatted fastball_avg_speed fastball_avg_spin
## 1                   27                 27                27
##   fastball_avg_break_x fastball_avg_break_z fastball_avg_break_z_induced
## 1                   27                   27                           27
##   fastball_avg_break fastball_range_speed n_breaking_formatted
## 1                 27                   27                   63
##   breaking_avg_speed breaking_avg_spin breaking_avg_break_x
## 1                 63                63                   63
##   breaking_avg_break_z breaking_avg_break_z_induced breaking_avg_break
## 1                   63                           63                63
##   breaking_range_speed n_offspeed_formatted offspeed_avg_speed
## 1                   78                  746                746
##   offspeed_avg_spin offspeed_avg_break_x offspeed_avg_break_z
## 1               748                  746                  746
##   offspeed_avg_break_z_induced offspeed_avg_break offspeed_range_speed
## 1                          746                746                840
```

```
# More data cleainging and adding feature `called_strike_and_whiff%`
stats <- stats %>%
  mutate(
    Name = str_replace_all(`last_name, first_name`, "([^,]+),\\s*(.+)", "\\2 \\1"),
    `CSW%` = (p_called_strike+p_swinging_strike)/p_total_pitches
  ) %>%
  drop_na(z_swing_miss_percent, oz_swing_miss_percent) %>%
  select(-c('last_name, first_name'))

# Joining Savant with Fangraphs data
joined_data <- stats %>%
  left_join(data, by = c('year' = 'Season', 'player_id' = 'PlayerId')) %>%
  arrange(year, TBF)

# Filling in NaN's with 0's for pitch usage, and deselecting colinear/non-useful data
joined_data <- joined_data %>%
  mutate(n_fastball_formatted = coalesce(n_fastball_formatted, 0),
         n_breaking_formatted = coalesce(n_breaking_formatted, 0),
         n_offspeed_formatted = coalesce(n_offspeed_formatted, 0)
         ) %>%
  select(-c(player_age, p_formatted_ip, z_swing_percent, oz_swing_percent, out_zone_perc
ent, in_zone_percent, fastball_avg_spin, fastball_avg_break_x, fastball_avg_break_z, fas
tball_avg_break, fastball_range_speed, breaking_avg_break_z, breaking_avg_break, breakin
g_range_speed, offspeed_avg_break_z, offspeed_avg_break, offspeed_range_speed, avg_best_
speed, avg_hyper_speed, pitch_count_offspeed, pitch_count_fastball, pitch_count_breakin
g, p_total_pitches, p_total_strike, p_called_strike, p_swinging_strike, p_total_swinging
_strike))
```

In order for us to include the lag (previous) season info for each player for a given year, we need to separate the data into their seasons and then join the previous season(s) data then calculate the difference in features between the years.

One feature that I also wanted to add was Tom Tango's Marcel projection tool (https://www.tangotiger.net/archives/stud0346.shtml (https://www.tangotiger.net/archives/stud0346.shtml)). His 5/4/3 projection weighs recent season performance and pulls an estimate `K%` towards the league average. This is a baseline prediction for projecting `K%` based on historical data. Adding this as a feature gives our model a bayesian prior that prevents the model from overweighting a lucky pitcher that had a good season.

```
#Separating data by year, adding a year suffix on the end of each feature
joined_data_2021 = joined_data %>% filter(year == 2021)
joined_data_2021 <- joined_data_2021 %>%
  rename_with(~ paste0(., "_21"), .cols = -c(player_id, year, Name.x, Name.y, Team))

joined_data_2022 = joined_data %>% filter(year == 2022)
joined_data_2022 <- joined_data_2022 %>%
  rename_with(~ paste0(., "_22"), .cols = -c(player_id, year, Name.x, Name.y, Team))

joined_data_2023 = joined_data %>% filter(year == 2023)
joined_data_2023 <- joined_data_2023 %>%
  rename_with(~ paste0(., "_23"), .cols = -c(player_id, year, Name.x, Name.y, Team))

joined_data_2024 = joined_data %>% filter(year == 2024)
joined_data_2024 <- joined_data_2024 %>%
  rename_with(~ paste0(., "_24"), .cols = -c(player_id, year, Name.x, Name.y, Team))

joined_data_2025 = joined_data %>% filter(year == 2025)
joined_data_2025 <- joined_data_2025 %>%
  rename_with(~ paste0(., "_25"), .cols = -c(player_id, year, Name.x, Name.y, Team))

# Joining 2022 & 2021 pitcher stats onto 2023 pitchers
stack_2023 <- joined_data_2023 %>%
  left_join(
    joined_data_2022 %>% select(-c(Name.x, Name.y, Team, year)),
    by = c('player_id')
  ) %>%
  left_join(
    joined_data_2021 %>% select(-c(Name.x, Name.y, Team, year)),
    by = c('player_id')
  )

# Getting feature columns from 2022 and 2021
base_cols <- colnames(joined_data_2022 %>% select(-c('player_id', 'year', 'Name.x', 'Nam
e.y', 'Team', 'Age_22')))
prev_cols <- str_replace(base_cols, "_22$", "_21")
diff_cols <- str_replace(base_cols, "_22$", "_diff")

# Calculating the difference for each feature between 2022 and 2021
stack_2023[diff_cols] <- stack_2023[base_cols] - stack_2023[prev_cols]

# Marcels projection of K% for 2023 season
stack_2023 <- stack_2023 %>%
  mutate(
    total_so = `K%_22` * TBF_22,
    Age_22 = coalesce(Age_22, Age_23 - 1),
    age_adj = if_else(Age_22 > 29, Age_22 * 0.006, Age_22 * 0.003)
  )
league_K <- sum(stack_2023$total_so, na.rm = TRUE) /
              sum(stack_2023$TBF_22, na.rm = TRUE)
stack_2023 <- stack_2023 %>%
  mutate(
```

```
    num = 5 * coalesce(`K%_22`, 0) * coalesce(TBF_22, 0) +
          4 * coalesce(`K%_21`, 0) * coalesce(TBF_21, 0) +
          0 +
          500 * league_K,
    den = 5 * coalesce(TBF_23, 0) +
          4 * coalesce(TBF_22, 0) +
          0 +
          500,
    marcels_22 = num / den * age_adj
  )

# Selecting only stats from previous season and the difference since the last season bef
ore that.
# Also adding a binary variable to signify new pitchers, as well as updating `Age` varia
ble.
stack_2023 <- stack_2023 %>%
  mutate(is_new_22 = if_else(is.na(`K%_22`), 1, 0)) %>%
  select(ends_with("_22"), ends_with("_diff"), `K%_23`, TBF_23) %>%
  rename_with(~ str_remove(., "_23$")) %>%
  rename_with(~ paste0("prev_", str_remove(., "_22$")), ends_with("_22"))

# Joining 2023 & 2022 pitcher stats onto 2024 pitchers
stack_2024 <- joined_data_2024 %>%
  left_join(
    joined_data_2023 %>% select(-c(Name.x, Name.y, Team, year)),
    by = c('player_id')
  ) %>%
  left_join(
    joined_data_2022 %>% select(-c(Name.x, Name.y, Team, year)),
    by = c('player_id')
  ) %>%
  left_join(
    joined_data_2021 %>% select(player_id, `K%_21`, TBF_21),
    by = c('player_id')
  )
# Getting feature columns from 2022 and 2021
base_cols <- colnames(joined_data_2023 %>% select(-c('player_id', 'year', 'Name.x', 'Nam
e.y', 'Team', 'Age_23')))
prev_cols <- str_replace(base_cols, "_23$", "_22")
diff_cols <- str_replace(base_cols, "_23$", "_diff")

# Calculating the difference for each feature between 2022 and 2021
stack_2024[diff_cols] <- stack_2024[base_cols] - stack_2024[prev_cols]

# Marcels projection of K% for 2024 season
stack_2024 <- stack_2024 %>%
  mutate(
    total_so = `K%_23` * TBF_23,
    Age_23 = coalesce(Age_23, Age_24 - 1),
    age_adj = if_else(Age_23 > 29, Age_23 * 0.006, Age_23 * 0.003)
  )
league_K <- sum(stack_2024$total_so, na.rm = TRUE) /
```

```
              sum(stack_2024$TBF_23, na.rm = TRUE)
stack_2024 <- stack_2024 %>%
  mutate(
    num = 5 * coalesce(`K%_23`, 0) * coalesce(TBF_23, 0) +
          4 * coalesce(`K%_22`, 0) * coalesce(TBF_22, 0) +
          3 * coalesce(`K%_21`, 0) * coalesce(TBF_21, 0) +
          500 * league_K,
    den = 5 * coalesce(TBF_23, 0) +
          4 * coalesce(TBF_22, 0) +
          3 * coalesce(TBF_21, 0) +
          500,
    marcels_23 = num / den * age_adj
  )

# Selecting only stats from previous season and the difference since the last season bef
ore that.
# Also adding a binary variable to signify new pitchers, as well as updating `Age` varia
ble.
stack_2024 <- stack_2024 %>%
  mutate(is_new_23 = if_else(is.na(`K%_23`), 1, 0)) %>%
  select(ends_with("_23"), ends_with("_diff"), `K%_24`, TBF_24) %>%
  rename_with(~ str_remove(., "_24$")) %>%
  rename_with(~ paste0("prev_", str_remove(., "_23$")), ends_with("_23"))

# Stacking two dataframes to create training data
train_data <- bind_rows(stack_2024, stack_2023)
```

Methodology

Data Constraints

As previously explained, the idea was to use a players previous season stats and difference in stats from the year before that to predict the current season `K%`. There were some problems that immediately came to my mind at first.

1. Positional Pitchers.

Some of these pitchers in our data are not pitchers, and are positional pitchers that pitched because they were in a blowout. Without further data description it was hard to identify and filter out the positional pitchers since even though they pitched in very little games, there were also real pitchers that pitched a little amount of games that either got injured or just didn't pitch a whole lot. I didn't want to filter out all pitchers that didn't pitch to say more than 50 batters for the sake of the real pitchers that also pitched to a less amount of batters. For example Seranthony Dominguez pitched to only 3 batters in 2021 and then pitched to 207 the next year, while maintaining a similar K% and Stuff+ for both years. I didn't want to lose the information that 2021 Seranthony could carry for the model. So Ijust left that data in there since it's unlikely there a huge sample of positional pitchers.

2. Pitchers with Small TBF.

Clearly not every pitcher pitches to the same amount of batters every year. Relief pitchers pitch to a much smaller sample size of batters and are usually in more high stakes situations. Although our strikeout metric is already standardized, pitchers that faced less batters may have an inflated strikeout rate since they don't pitch to as

many batters. So to deal with that, I decided to use `total_batters_faced` as weights for each pitcher, where our model will weigh down those pitchers with inflated strikeout rates. In my implementation, the weights are set as $sqrt(TBF)$, which will prevent low sample pitchers with inflated/random `K%` from skewing our data. This also assists with the positional pitcher issue. To note, we do have the marcels metric as a feature as a projection already but even that isn't perfect so this will assist on top of that.

3. Pitchers with incomplete arsenals.

For pitchers that don't have a fastball/breaking/offspeed pitch in their arsenal, the data by default leaves it as NaN, which we don't want. By coalescing the usage to 0% instead of having NaN,it allows us to place those pitchers with other pitchers who don't use that pitch on a similar level.

Model Training

I had two different model types to test. A linear model and tree-based model. I trained a GLM model at first on the data as a baseline model. A GLM model can capture the non-linear relationship between variables. However I wanted totrain an LightGBM and XGBoost model, as a tree based model is better for dealing with non-linear relationships, handles outliers well, and handles missing data that we want to keep.

GLM Model

```
set.seed(8)

glm_train <- train_data
# 80/20 train test split
train_indices <- sample(1:nrow(glm_train), size = 0.8 * nrow(glm_train))
train_df <- glm_train[train_indices, ]
test_df  <- glm_train[-train_indices, ]

# Model training using GLM
base_model <- glm(`K%` ~ ., data = train_df, family = quasibinomial(link = "logit"))

# Predict on test data
test_df$predicted_so <- predict(base_model, newdata = test_df, type = "response")

# Find difference in error between pred and actual
errors <- test_df$`K%` - test_df$predicted_so

print(paste("MSE:",
            sprintf("%.4f", sqrt(mean(errors^2, na.rm=TRUE)))))
```

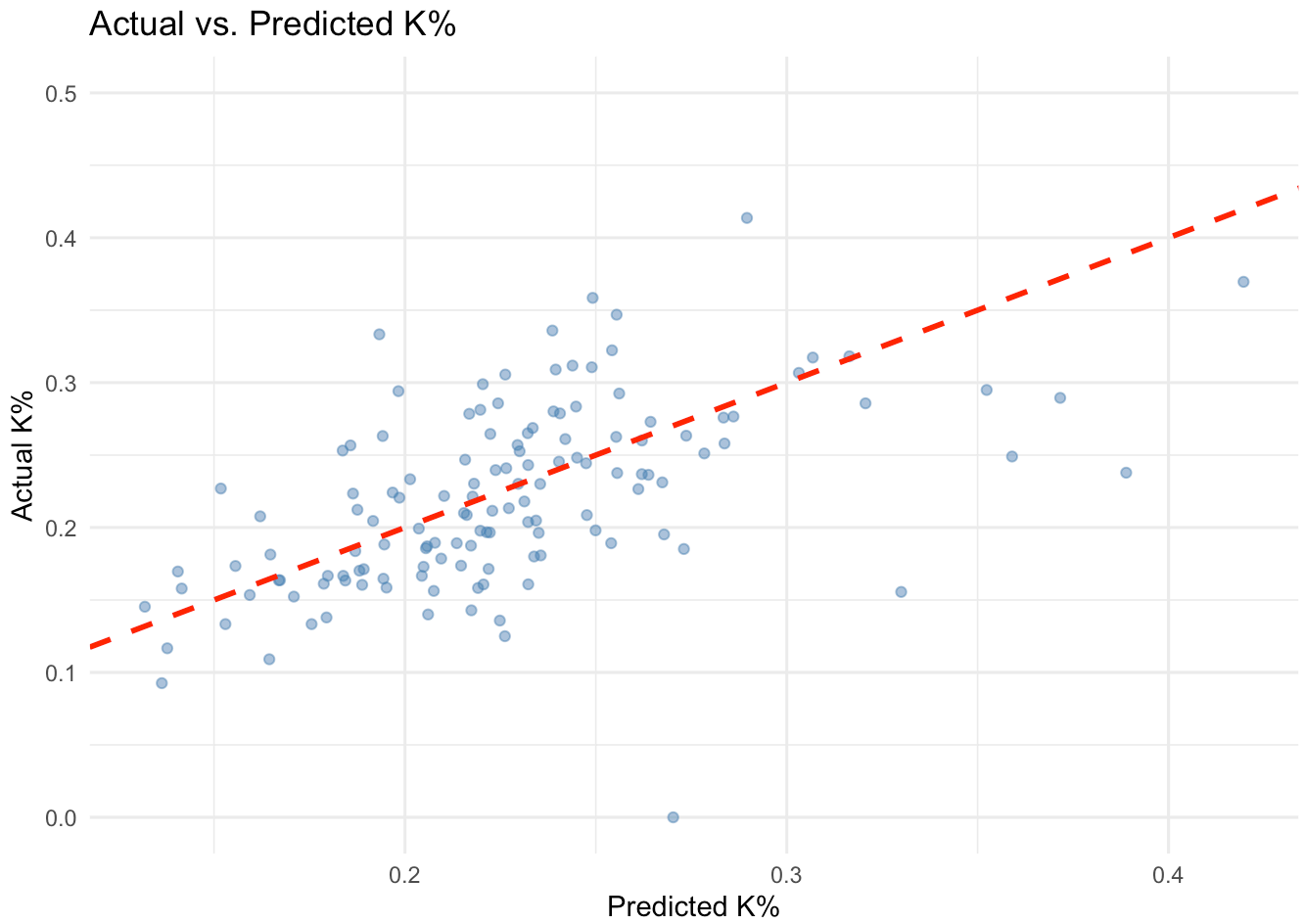
```
## [1] "MSE: 0.0556"
```

```
print(paste("R^2:",
            sprintf("%.4f", cor(test_df$predicted_so, test_df$`K%`, use = "complete.ob
s")^2)))
```

```
## [1] "R^2: 0.2827"
```

```
ggplot(test_df, aes(x = `predicted_so`, y = `K%`)) +
  geom_point(alpha = 0.5, color = "steelblue") +
  geom_abline(slope = 1, intercept = 0, color = "red", linetype = "dashed", linewidth =
1) +
  labs(
    title = "Actual vs. Predicted K%",
    x = "Predicted K%",
    y = "Actual K%"
  ) +
  theme_minimal()
```

```
## Warning: Removed 185 rows containing missing values or values outside the scale range
## (`geom_point()`).
```



We can see that our base model has a MSE of 0.055 and R^2 of 28%, where our model explains ~28% of variance in our data. However our model does not work for instances with NaN's and we cannot replace all NaN's or drop those rows of data. So we cannot use this model.

LGB Model

```
set.seed(9)

# 80/20 train test split
train_indices <- sample(1:nrow(train_data), size = 0.8 * nrow(train_data))
train_df <- train_data[train_indices, ]
test_df  <- train_data[-train_indices, ]
feature_cols <- setdiff(names(train_data), c('K%', 'TBF'))

X_train <- as.matrix(train_df[, feature_cols])
y_train <- train_df$`K%`
w_train <- sqrt(train_df$TBF)

X_test  <- as.matrix(test_df[, feature_cols])
y_test  <- test_df$`K%`
w_test  <- sqrt(test_df$TBF)

dtrain <- lgb.Dataset(data = X_train, label = y_train, weight = w_train)
dtest  <- lgb.Dataset(data = X_test,  label = y_test,  weight = w_test, reference = dtra
in)

# Initialize parameters
params <- list(
  objective          = "regression",
  metric             = "rmse",
  learning_rate      = 0.01,
  num_leaves         = 7,
  min_data_in_leaf   = 40,
  feature_fraction   = 0.65,
  bagging_fraction   = 0.80,
  bagging_freq       = 1,
  lambda_l1          = 0.5,
  lambda_l2          = 1.0
)

valids <- list(train = dtrain, test = dtest)

# Train LGB model
model_lgb <- lgb.train(
  params               = params,
  data                 = dtrain,
  nrounds              = 1000,
  valids               = valids,
  early_stopping_rounds = 50,
  verbose              = 1
)

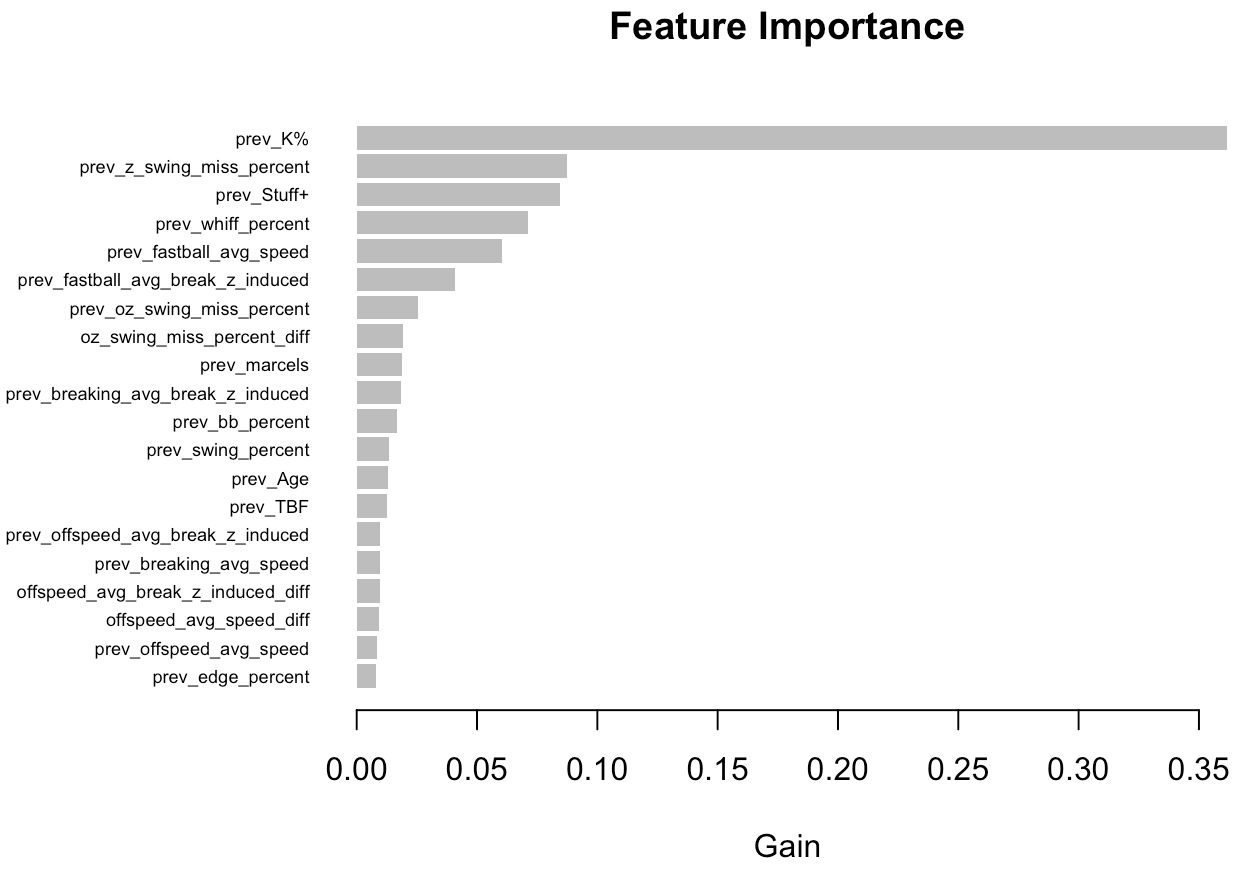
# Predict on test set
test_df$`pred_K%` <- predict(model_lgb, X_test)

print(summary(test_df$`pred_K%`))
```

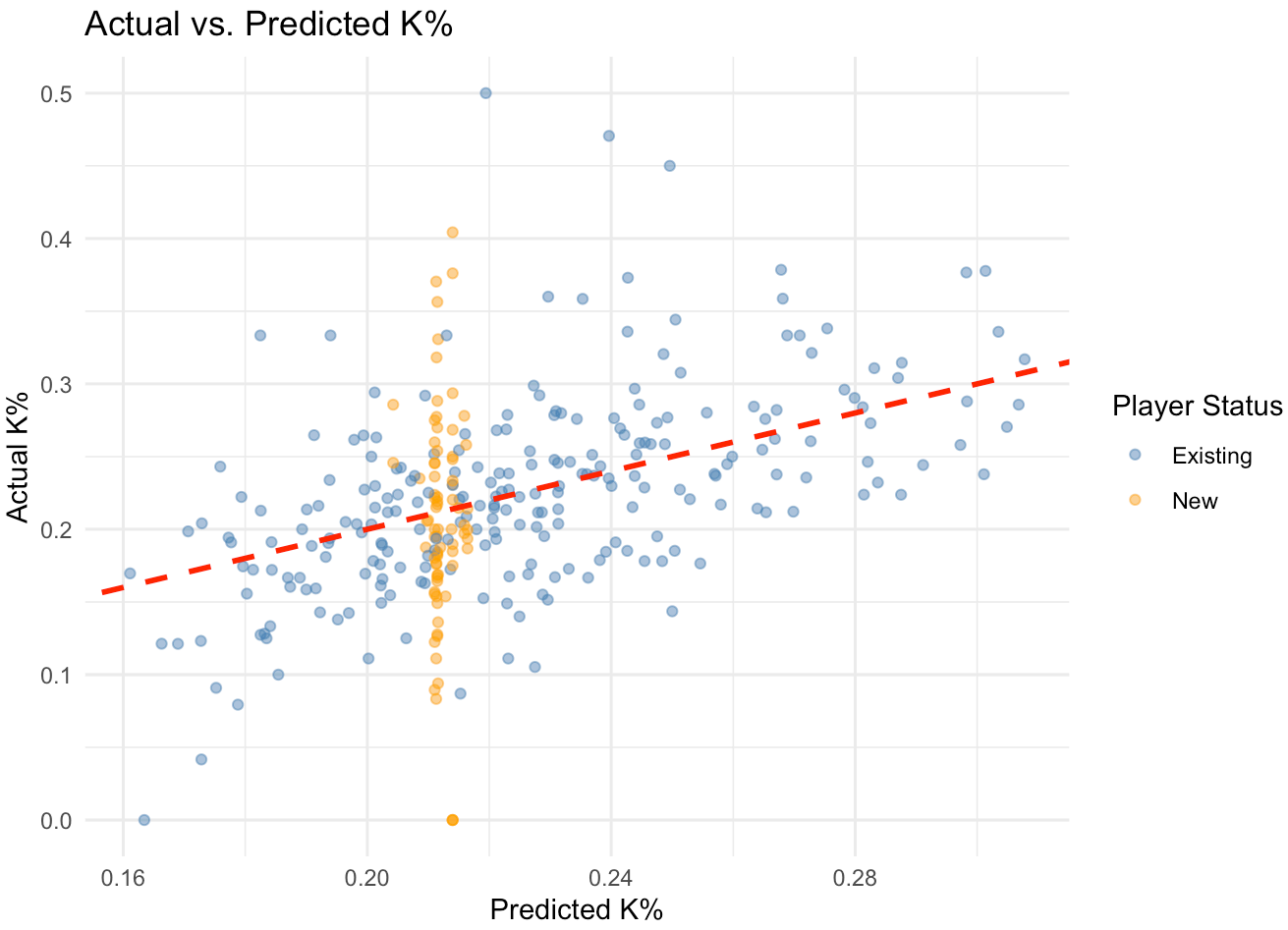
```
rmse_val <- sqrt(mean((test_df$`K%` - test_df$`pred_K%`)^2))
r2_val   <- cor(test_df$`K%`, test_df$`pred_K%`)^2

print(paste0("Test RMSE: ", sprintf("%.4f", rmse_val)))
print(paste0("Test R²:   ", sprintf("%.4f", r2_val)))

# Feature Importance
importance <- lgb.importance(model_lgb, percentage = TRUE)
lgb.plot.importance(importance, top_n = 20, measure = "Gain")
```



```
ggplot(test_df, aes(x = `pred_K%`, y = `K%`)) +
  geom_point(aes(color = factor(prev_is_new)), alpha = 0.5) +
  scale_color_manual(
    values = c("0" = "steelblue", "1" = "orange"),
    labels = c("0" = "Existing", "1" = "New"),
    name = "Player Status"
  ) +
  geom_abline(slope = 1, intercept = 0, color = "red", linetype = "dashed", linewidth =
1) +
  labs(
    title = "Actual vs. Predicted K%",
    x = "Predicted K%",
    y = "Actual K%"
  ) +
  theme_minimal()
```



In this model, our test results give us a RMSE of 0.06 and R^2 of 22%. From our feature importance plot we can see that last seasons strikeout rate, zone swing & miss%, whiff%, fastball avg speed, and stuff+ are our best predictors. However when looking at our plot of actual vs predicted K% , we can see that we have a handful of new pitchers that are skewing our predictions. Which makes sense because without previous season data our model just makes normal random guesses for new pitchers, which seem to average out at ~22.5%. So if we take away those new pitchers, we can see how our model does on existing pitchers.

```
# removing new pitchers
no_rooks <- test_df %>% filter(!is.na(prev_TBF))

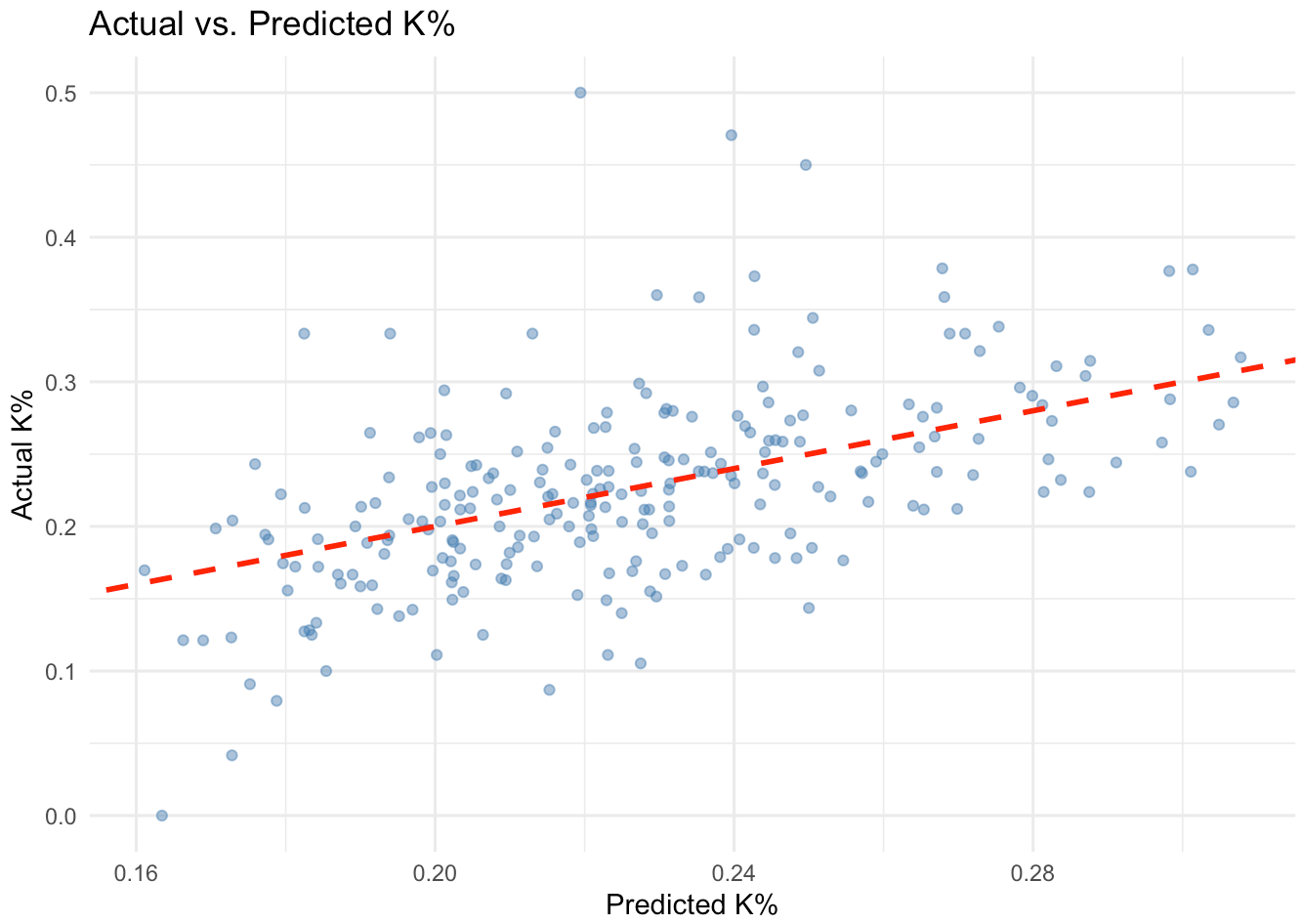
# Test RMSE and R^2
print(paste0("Test RMSE: ", sprintf("%.4f", sqrt(mean((no_rooks$`K%` - no_rooks$`pred_K%`)^2)))))
```

```
## [1] "Test RMSE: 0.0566"
```

```
print(paste0("Test R²: ", sprintf("%.4f", cor(no_rooks$`K%`, no_rooks$`pred_K%`, use = "complete.obs")^2)))
```

```
## [1] "Test R²: 0.2979"
```

```
ggplot(no_rooks, aes(x = `pred_K%`, y = `K%`)) +
  geom_point(alpha = 0.5, color = "steelblue") +
  geom_abline(slope = 1, intercept = 0, color = "red", linetype = "dashed", linewidth = 1) +
  labs(
    title = "Actual vs. Predicted K%",
    x = "Predicted K%",
    y = "Actual K%"
  ) +
  theme_minimal()
```



Now we get an improved RSME of 0.0566 and R^2 of 29%.

XGBoost Model

```
X_train <- as.matrix(train_df[, feature_cols])
y_train <- train_df$`K%`
w_train <- sqrt(train_df$TBF)

X_test  <- as.matrix(test_df[, feature_cols])
y_test  <- test_df$`K%`
w_test  <- sqrt(test_df$TBF)

dtrain <- xgb.DMatrix(data = X_train, label = y_train, weight = w_train)
dtest  <- xgb.DMatrix(data = X_test,  label = y_test,  weight = w_test)

params <- list(
  objective        = "reg:squarederror", # Robust to extreme fluke reliever seasons
  eval_metric      = "rmse",
  eta              = 0.02,                  # Low learning rate for stable convergence
  max_depth        = 3,                     # Shallow depth prevents fitting single-sea
son noise
  min_child_weight = 10,                    # Enforces large sample support per leaf
  subsample        = 0.75,                  # Row subsampling
  colsample_bytree = 0.75,                  # Feature subsampling
  lambda           = 5.0,                   # L2 regularization (Ridge penalty)
  alpha            = 0.5                    # L1 regularization (Lasso penalty)
)

watchlist <- list(train = dtrain, test = dtest)

model_xgb <- xgb.train(
  params               = params,
  data                 = dtrain,
  nrounds              = 1000,
  watchlist            = watchlist,
  early_stopping_rounds = 40,
  print_every_n        = 20
)
```

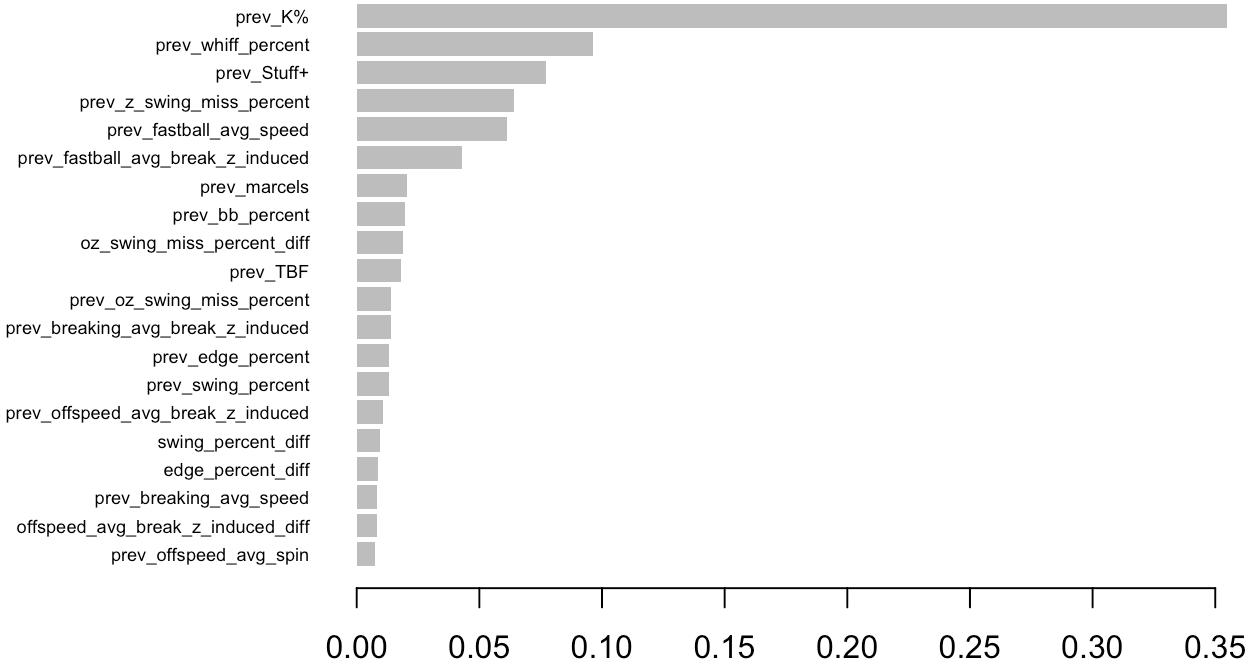
```
## Warning in throw_err_or_depr_msg("Parameter '", match_old, "' has been renamed
## to '", : Parameter 'watchlist' has been renamed to 'evals'. This warning will
## become an error in a future version.
```

```
test_df$`pred_K%` <- predict(model_xgb, dtest)

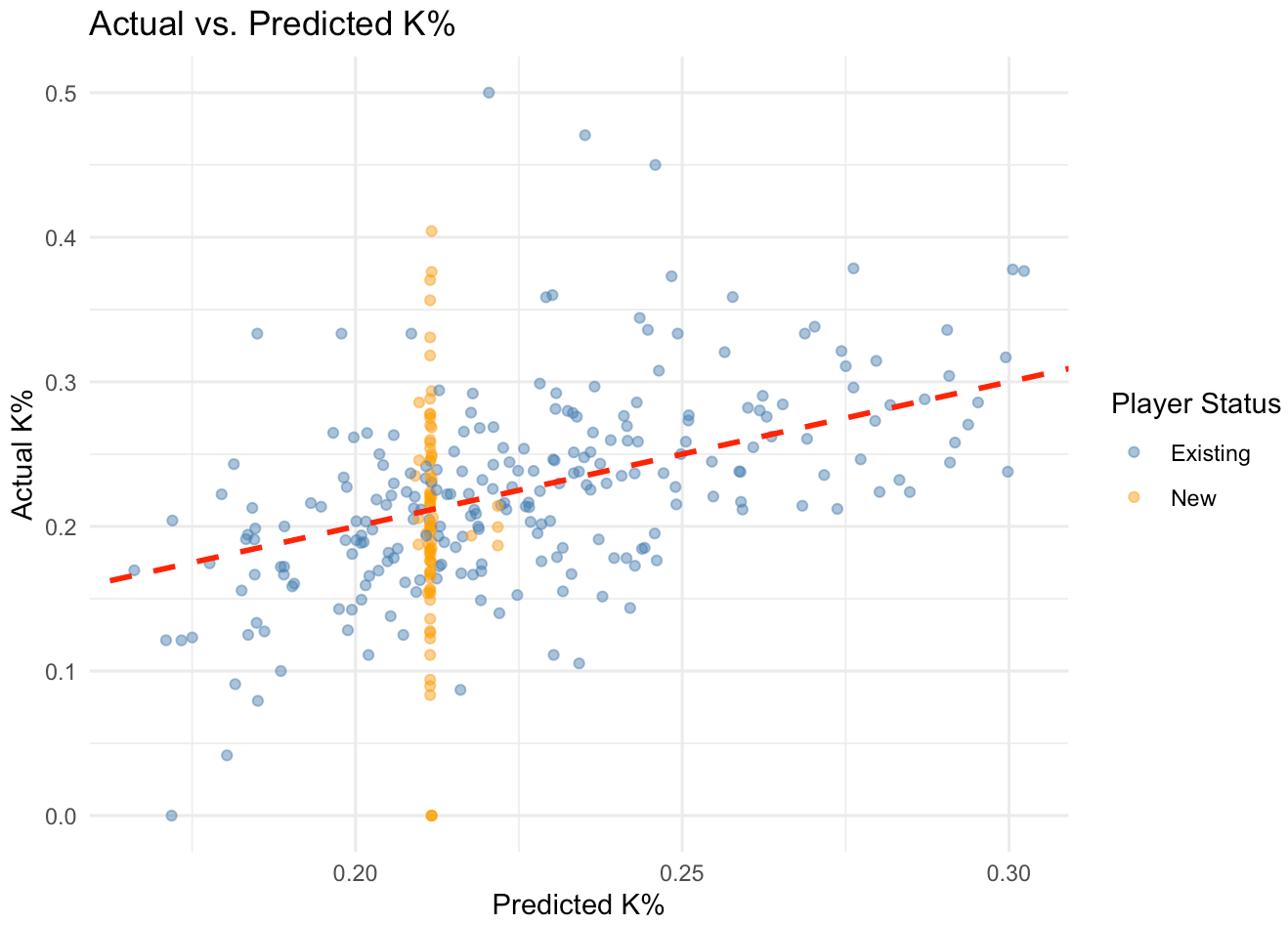
rmse_all <- sqrt(mean((test_df$`K%` - test_df$`pred_K`)^2))
r2_all   <- cor(test_df$`K%`, test_df$`pred_K`)^2

print(paste0("Test RMSE: ", sprintf("%.4f", rmse_all)))
print(paste0("Test R²:   ", sprintf("%.4f", r2_all)))

importance_matrix <- xgb.importance(feature_names = feature_cols, model = model_xgb)
xgb.plot.importance(importance_matrix = importance_matrix, top_n = 20)
```



```
ggplot(test_df, aes(x = `pred_K%`, y = `K%`)) +
  geom_point(aes(color = factor(prev_is_new)), alpha = 0.5) +
  scale_color_manual(
    values = c("0" = "steelblue", "1" = "orange"),
    labels = c("0" = "Existing", "1" = "New"),
    name = "Player Status"
  ) +
  geom_abline(slope = 1, intercept = 0, color = "red", linetype = "dashed", linewidth =
1) +
  labs(
    title = "Actual vs. Predicted K%",
    x = "Predicted K%",
    y = "Actual K%"
  ) +
  theme_minimal()
```



For our XGBoost model, our test results give us a RMSE of 0.06 and R^2 of 22%. From our feature importance plot we can see that last seasons strikeout rate, zone swing & miss%, whiff%, stuff+, and fastball avg break/speed are our best predictors. However we see the similar issue of new pitchers that skew our predictions. We can similarly take away those new pitchers and see how our model does on existing pitchers.

```
# removing new pitchers
no_rooks <- test_df %>% filter(!is.na(prev_TBF))

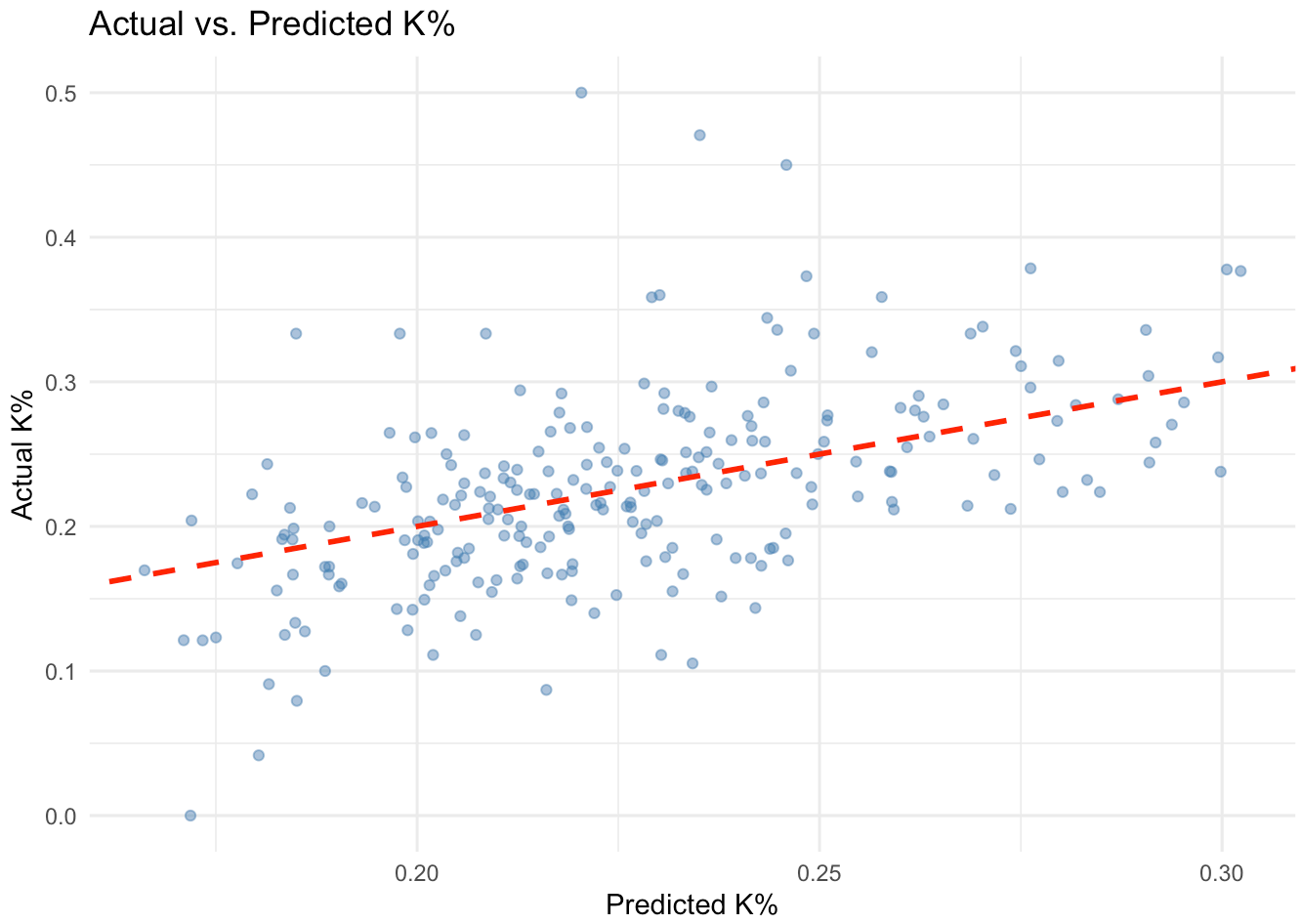
# Test RSME and R^2
print(paste0("Test RMSE: ", sprintf("%.4f", sqrt(mean((no_rooks$`K%` - no_rooks$`pred_K%
`)^2)))))
```

```
## [1] "Test RMSE: 0.0571"
```

```
print(paste0("Test R²: ", sprintf("%.4f", cor(no_rooks$`K%`, no_rooks$`pred_K%`, use =
"complete.obs")^2)))
```

```
## [1] "Test R²: 0.2917"
```

```
ggplot(no_rooks, aes(x = `pred_K%`, y = `K%`)) +
  geom_point(alpha = 0.5, color = "steelblue") +
  geom_abline(slope = 1, intercept = 0, color = "red", linetype = "dashed", linewidth =
1) +
  labs(
    title = "Actual vs. Predicted K%",
    x = "Predicted K%",
    y = "Actual K%"
  ) +
  theme_minimal()
```



Now we get an RSME of 0.0571 and R^2 of 29%.

XGBoost slightly outperforms LightGBM so we will use the XGB model.

Analysis

Testing on 2025 data

Using the same setup and feature selection, we can test our model on 2025 data.

```
# Joining 2024 & 2023 pitcher stats onto 2025 pitchers
test_2025 <- joined_data_2025 %>%
  left_join(
    joined_data_2024 %>% select(-c(Name.x, Name.y, Team, year)),
    by = c('player_id')
  ) %>%
  left_join(
    joined_data_2023 %>% select(-c(Name.x, Name.y, Team, year)),
    by = c('player_id')
  ) %>%
  left_join(
    joined_data_2022 %>% select(player_id, `K%_22`, TBF_22),
    by = c('player_id')
  )

# Getting feature columns from 2024 and 2023
base_cols <- colnames(joined_data_2024 %>% select(-c('player_id', 'year', 'Name.x', 'Nam
e.y', 'Team', 'Age_24')))
prev_cols <- str_replace(base_cols, "_24$", "_23")
diff_cols <- str_replace(base_cols, "_24$", "_diff")


# Calculating the difference for each feature between 2022 and 2021
test_2025[diff_cols] <- test_2025[base_cols] - test_2025[prev_cols]

# Marcels projection of K% for 2025 season
test_2025 <- test_2025 %>%
  mutate(
    total_so = `K%_24` * TBF_24,
    Age_24 = coalesce(Age_24, Age_25 - 1),
    age_adj = if_else(Age_24 > 29, Age_24 * 0.006, Age_24 * 0.003)
  )
league_K <- sum(test_2025$total_so, na.rm = TRUE) /
              sum(test_2025$TBF_24, na.rm = TRUE)
test_2025 <- test_2025 %>%
  mutate(
    num = 5 * coalesce(`K%_24`, 0) * coalesce(TBF_24, 0) +
          4 * coalesce(`K%_23`, 0) * coalesce(TBF_23, 0) +
          3 * coalesce(`K%_22`, 0) * coalesce(TBF_22, 0) +
          500 * league_K,
    den = 5 * coalesce(TBF_24, 0) +
          4 * coalesce(TBF_23, 0) +
          3 * coalesce(TBF_22, 0) +
          500,
    marcels_24 = num / den * age_adj
  )

# Selecting only stats from previous season and the difference since the last season bef
ore that.
# Also adding a binary variable to signify new pitchers, as well as updating `Age` varia
ble.
test_2025 <- test_2025 %>%
```

```
  mutate(is_new_24 = if_else(is.na(`K%_24`), 1, 0)) %>%
  select(player_id, year, Name = Name.x, Team, Age_25, `Stuff+_25`, ends_with("_24"), en
ds_with("_diff"), `K%_25`, TBF_25) %>%
  rename_with(~ str_remove(., "_25$")) %>%
  rename_with(~ paste0("prev_", str_remove(., "_24$")), ends_with("_24"))

# Initialize Test Dataset
feature_cols <- setdiff(names(test_2025), c('player_id', 'year', 'Name', 'Team', 'Age',
'K%', 'TBF', 'Stuff+'))
X_test  <- as.matrix(test_2025[, feature_cols])

# Predict on test dataset using LGB model
test_2025$`pred_K%` <- predict(model_xgb, X_test)

# Test RSME and R^2
rmse_val <- sqrt(mean((test_2025$`K%` - test_2025$`pred_K%`)^2))
r2_val   <- cor(test_2025$`K%`, test_2025$`pred_K%`)^2

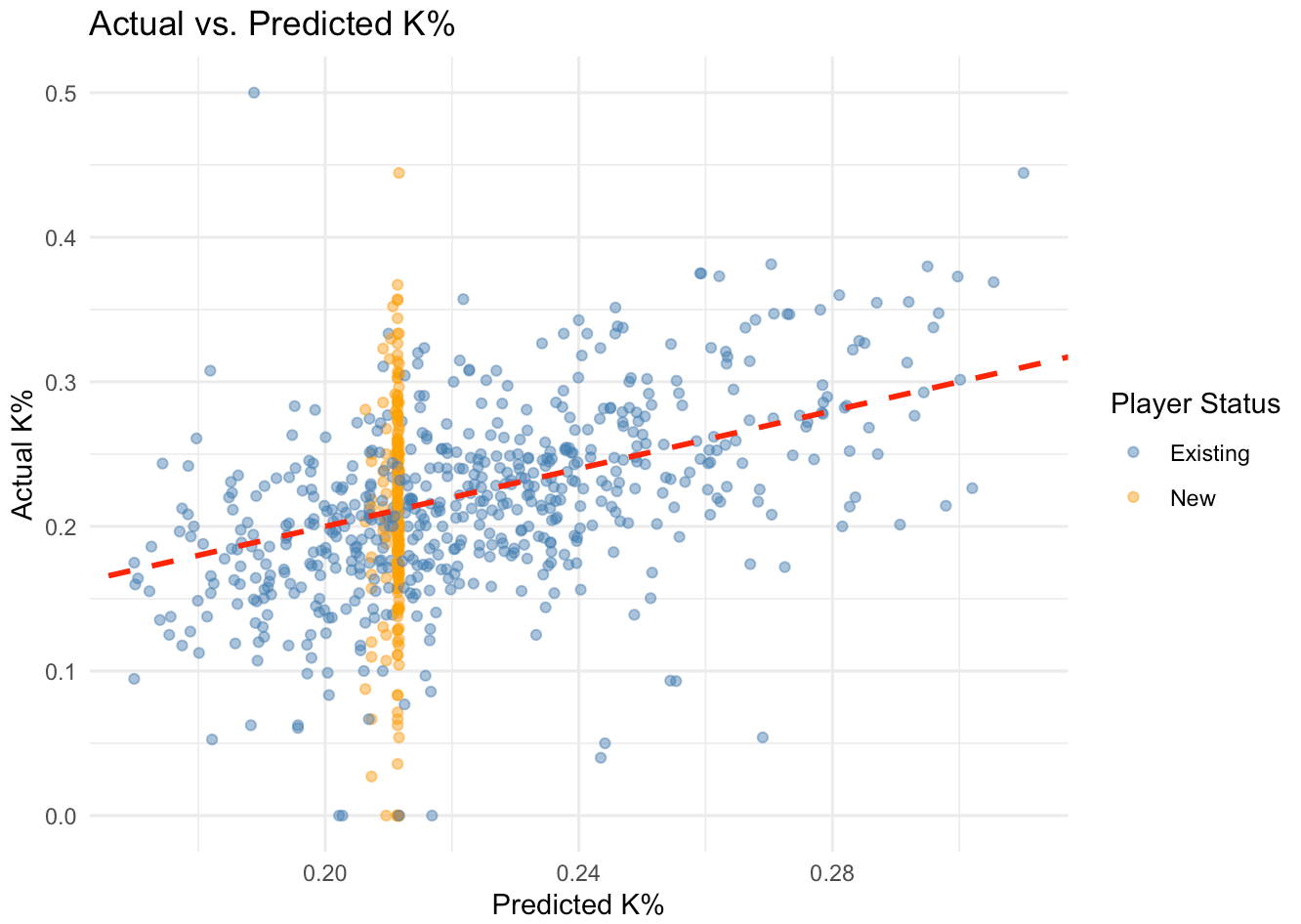
print(paste0("Test RMSE: ", sprintf("%.4f", rmse_val)))
```

```
## [1] "Test RMSE: 0.0623"
```

```
print(paste0("Test R²:   ", sprintf("%.4f", r2_val)))
```

```
## [1] "Test R²:   0.1855"
```

```
ggplot(test_2025, aes(x = `pred_K%`, y = `K%`)) +
  geom_point(aes(color = factor(prev_is_new)), alpha = 0.5) +
  scale_color_manual(
    values = c("0" = "steelblue", "1" = "orange"),
    labels = c("0" = "Existing", "1" = "New"),
    name = "Player Status"
  ) +
  geom_abline(slope = 1, intercept = 0, color = "red", linetype = "dashed", linewidth =
1) +
  labs(
    title = "Actual vs. Predicted K%",
    x = "Predicted K%",
    y = "Actual K%"
  ) +
  theme_minimal()
```



Our model gives us a RMSE of 0.06 and R^2 of 18% on our 2025 test data.

```
# removing new pitchers
no_rooks <- test_2025 %>% filter(prev_is_new == 0)

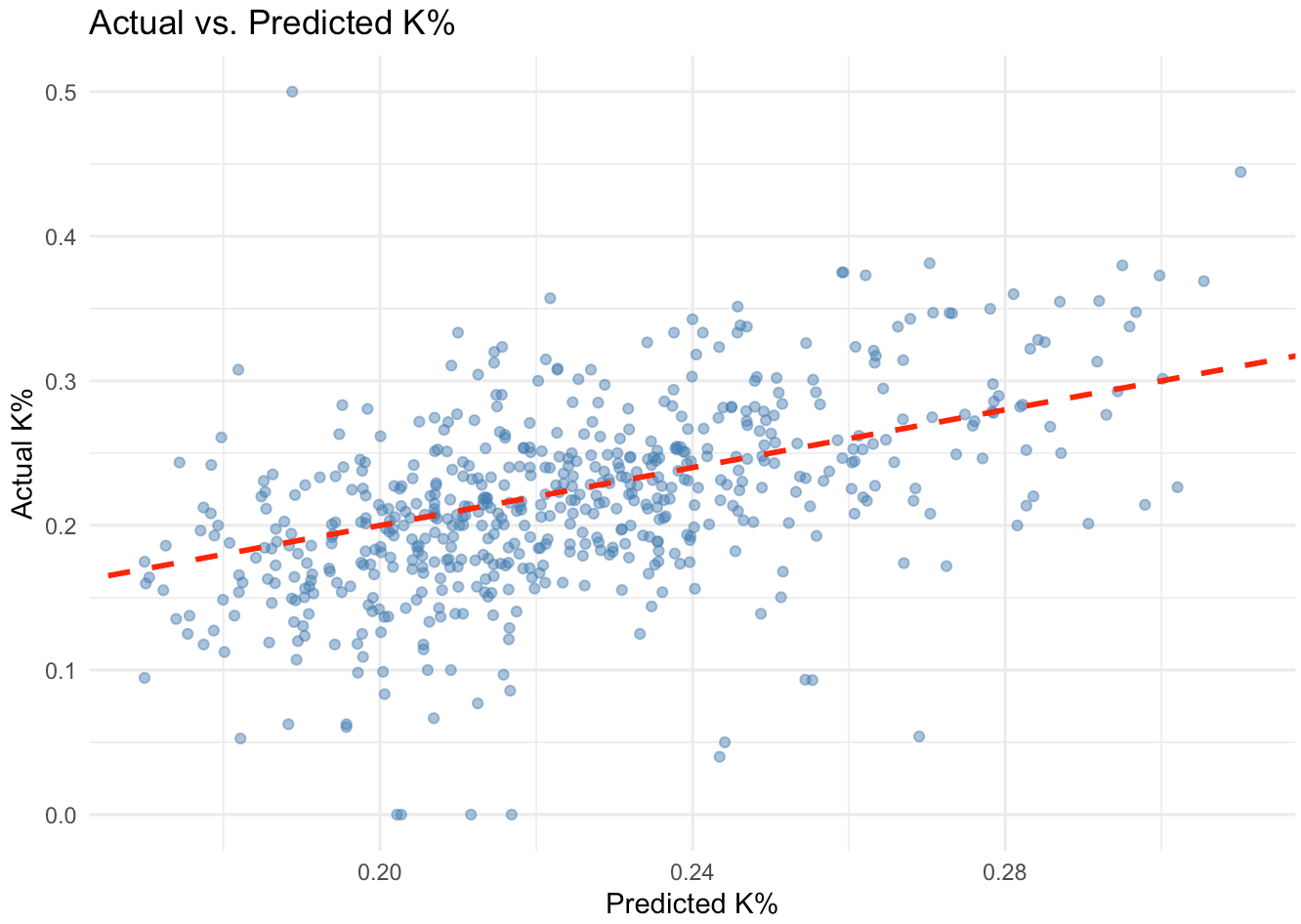
# Test RSME and R^2
print(paste0("Test RMSE: ", sprintf("%.4f", sqrt(mean((no_rooks$`K%` - no_rooks$`pred_K%
`)^2)))))
```

```
## [1] "Test RMSE: 0.0555"
```

```
print(paste0("Test R²: ", sprintf("%.4f", cor(no_rooks$`K%`, no_rooks$`pred_K%`, use =
"complete.obs")^2)))
```

```
## [1] "Test R²: 0.2752"
```

```
ggplot(no_rooks, aes(x = `pred_K%`, y = `K%`)) +
  geom_point(alpha = 0.5, color = "steelblue") +
  geom_abline(slope = 1, intercept = 0, color = "red", linetype = "dashed", linewidth =
1) +
  labs(
    title = "Actual vs. Predicted K%",
    x = "Predicted K%",
    y = "Actual K%"
  ) +
  theme_minimal()
```



On existing pitchers, our model has an RSME of 0.05 and R^2 of 27% on existing pitchers.

```
test_2025 <- test_2025 %>%
  mutate(pred_diff = `pred_K%` - `K%`) %>%
  select(player_id, year, Name, Team, TBF, `Stuff+`, Age, `K%`, `pred_K%`, pred_diff, pr
ev_is_new) %>%
  arrange(desc(`pred_K%`))
```

Results

These are the top 5 projected pitchers by strikeout rate for 2025 that our model predicts:

```
test_2025 %>% arrange(desc(`pred_K%`)) %>% slice(1:5)
```

```
##   player_id year             Name  Team TBF   Stuff+ Age        K%   pred_K%
## 1    695243 2025     Mason Miller - - - 234 122.8171  26 0.4444444 0.3101327
## 2    623352 2025       Josh Hader   HOU 206 118.2213  31 0.3689320 0.3054197
## 3    656629 2025   Michael Kopech   LAD  53 117.6570  29 0.2264151 0.3020467
## 4    682254 2025 Mason Montgomery   TBR 209 124.7507  25 0.3014354 0.3001525
## 5    547973 2025  Aroldis Chapman   BOS 228 107.7259  37 0.3728070 0.2997501
##      pred_diff prev_is_new
## 1 -0.134311758           0
## 2 -0.063512386           0
## 3  0.075631656           0
## 4 -0.001282929           0
## 5 -0.073056960           0
```

These are the bottom 5 projected pitchers by strikeout rate for 2025 that our model predicts:

```
test_2025 %>% arrange(`pred_K%`) %>% slice(1:5)
```

```
##   player_id year            Name Team TBF   Stuff+ Age         K%   pred_K%
## 1    519008 2025  T.J. McFarland  ATH  74 85.98721  36 0.09459459 0.1698889
## 2    607536 2025   Kyle Freeland  COL 709 94.63670  32 0.17489422 0.1698998
## 3    592836 2025  Taijuan Walker  PHI 538 89.98278  32 0.15985130 0.1700509
## 4    543294 2025  Kyle Hendricks  LAA 695 82.52906  35 0.16402878 0.1704859
## 5    678368 2025 Valente Bellozo  MIA 348 94.23303  25 0.15517241 0.1722951
##      pred_diff prev_is_new
## 1  0.075294353           0
## 2 -0.004994414           0
## 3  0.010199589           0
## 4  0.006457089           0
## 5  0.017122698           0
```

These are the top 5 closest predictions our model makes to the actual strikeout rate:

```
test_2025 %>% slice_min(abs(pred_diff), n = 5)
```

```
##   player_id year             Name  Team TBF   Stuff+ Age        K%   pred_K%
## 1    677960 2025    Ryan Weathers   MIA 166 102.7455  25 0.2228916 0.2229347
## 2    502085 2025  David Robertson   PHI  78 106.9594  40 0.2820513 0.2819155
## 3    641302 2025  Tyler Alexander - - - 423 100.1870  30 0.1938534 0.1940347
## 4    683004 2025      Jack Leiter   TEX 646 107.0108  25 0.2291022 0.2293851
## 5    684320 2025 Yariel Rodríguez   TOR 298 101.2351  28 0.2214765 0.2211872
##       pred_diff prev_is_new
## 1  0.0000431678           0
## 2 -0.0001357941           0
## 3  0.0001812507           0
## 4  0.0002829676           0
## 5 -0.0002892910           0
```

These are the top 5 furthest predictions our model makes to the actual strikeout rate:

```
test_2025 %>% slice_max(abs(pred_diff), n = 5)
```

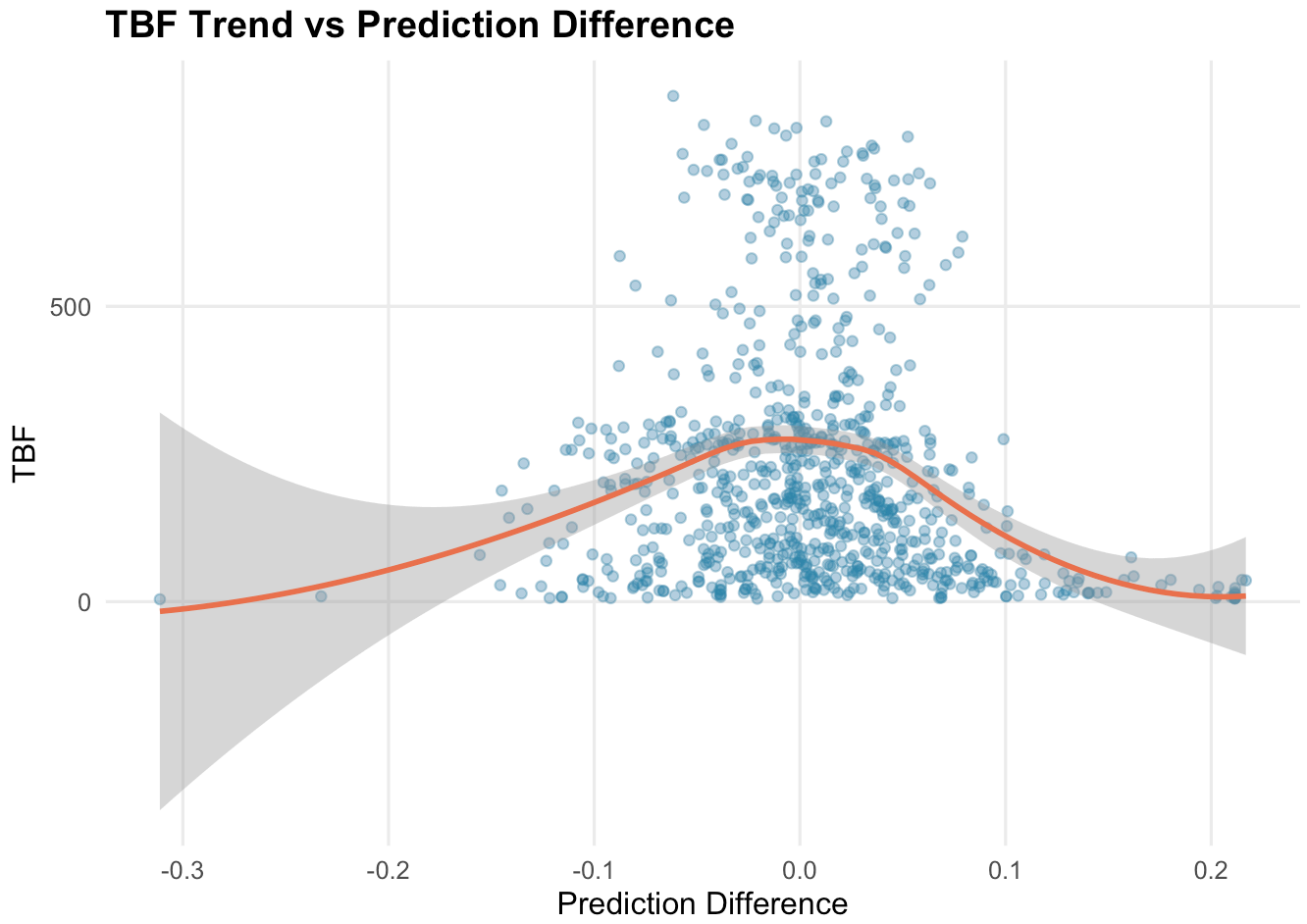
```
##   player_id year            Name Team TBF    Stuff+ Age         K%   pred_K%
## 1    656458 2025 Lucas Gilbreath  COL   4  82.36333  29 0.50000000 0.1887896
## 2    664744 2025     Jose Espada  BAL   9  83.57241  28 0.44444444 0.2116453
## 3    666619 2025  Gregory Santos  SEA  36 102.99385  25 0.00000000 0.2168526
## 4    700187 2025     Troy Taylor  SEA  37 102.90970  23 0.05405405 0.2689997
## 5    607179 2025    Jordan Weems  HOU  23  91.97245  32 0.00000000 0.2116590
##    pred_diff prev_is_new
## 1 -0.3112104           0
## 2 -0.2327991           1
## 3  0.2168526           0
## 4  0.2149456           0
## 5  0.2116590           0
```

If you notice for rows with a higher difference in prediction vs actual, we can see the TBF values are low. Meanwhile for the ones with low difference in pred vs actual, the TBF are mostly high. We can look at the trend in TBF values against the difference in prediction rates for `K%` :

```
TBF_trend <- test_2025 %>%
  arrange(pred_diff)

ggplot(TBF_trend, aes(x = pred_diff, y = TBF)) +
  geom_point(alpha = 0.4, color = "#2E86AB", size = 1.5) +
  geom_smooth(method = "loess", color = "#E9724C", se = TRUE, linewidth = 1) +
  labs(
    title = "TBF Trend vs Prediction Difference",
    x = "Prediction Difference",
    y = "TBF"
  ) +
  theme_minimal(base_size = 12) +
  theme(
    panel.grid.minor = element_blank(),
    plot.title = element_text(face = "bold")
  )
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



This plot validates our suspicion. The lower the TBF, the higher the difference is in our prediction. So lets take a closer look at the 'non-rookies' (just players who weren't in the league previously). Using LLM's assistance we are able to code a plot to view our overall results and how our model performs on each pitcher. X-axis is the difference between predicted and actual `K%` so the closer to 0 the better. And the Y-axis is the predicted `K%` .

```
# Filter to only non-new players
no_rooks <- test_2025 %>% filter(prev_is_new == 0)

# Get select players to label on plot for each quadrant
label_points <- bind_rows(
  # Top left (Under-predicted best pitchers)
  no_rooks %>%
    filter(pred_diff < 0, `pred_K%` > 0.225) %>%
    slice_max(`pred_K%`, n = 3),

  # Top right (Over-predicted best pitchers)
  no_rooks %>%
    filter(pred_diff > 0, `pred_K%` > 0.225) %>%
    slice_max(`pred_K%`, n = 3),

  # Bottom left (Under-predicted worst pitchers)
  no_rooks %>%
    filter(pred_diff < 0, `pred_K%` < 0.225) %>%
    slice_min(pred_diff, n = 3),

  # Bottom right (Over-predicted worst pitchers)
  no_rooks %>%
    filter(pred_diff > 0, `pred_K%` < 0.2) %>%
    slice_max(pred_diff, n = 3),

  # Midde (Accurately predicted)
  no_rooks %>%
    slice_min(abs(pred_diff), n = 3)
)

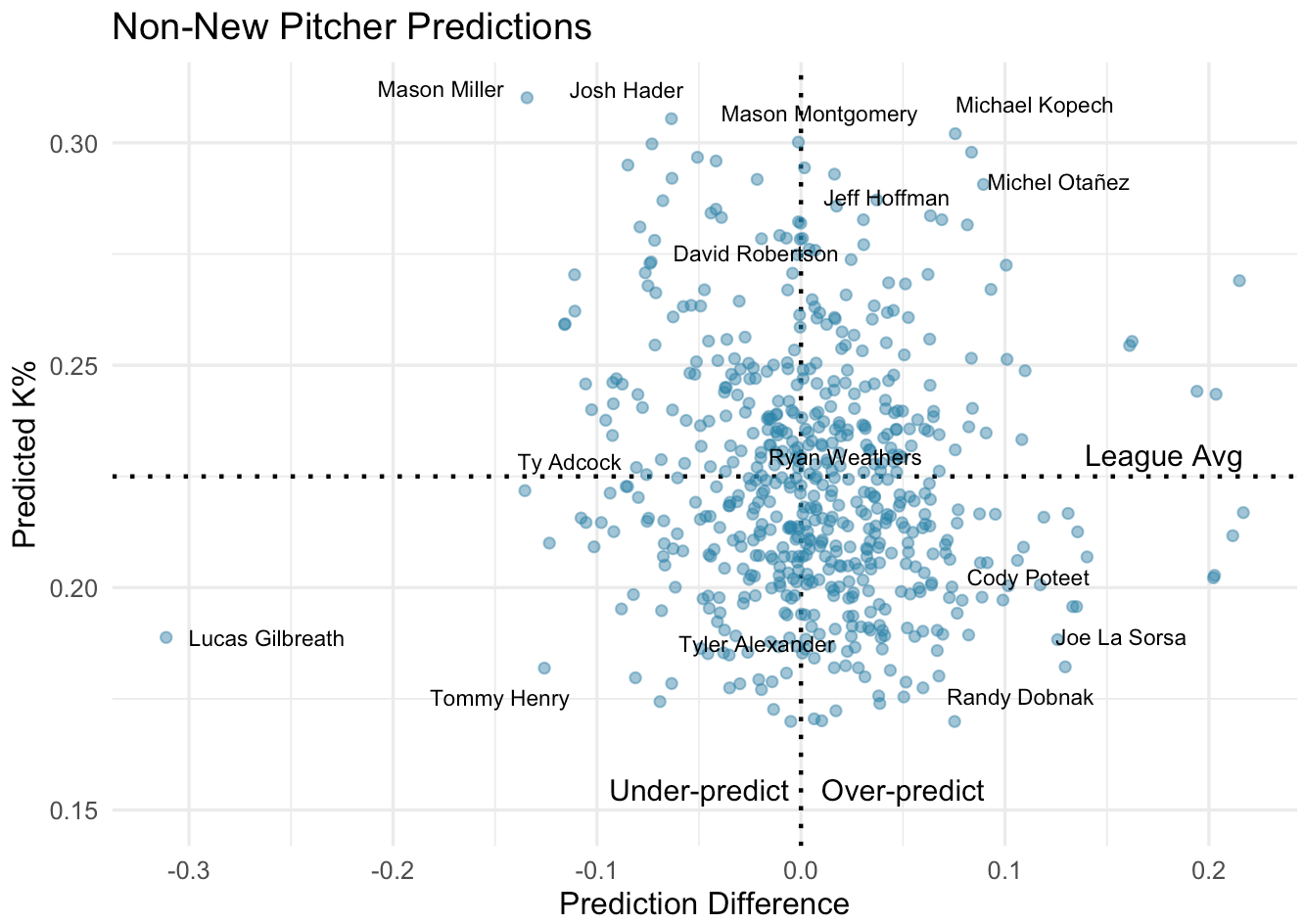
# Create visual
ggplot(no_rooks, aes(x = pred_diff, y = `pred_K%`)) +
  geom_vline(
    xintercept = 0,
    linetype = "dotted",
    linewidth = 0.8
  ) +
  geom_hline(
    yintercept = 0.225,
    linetype = "dotted",
    linewidth = 0.8
  ) +
  geom_point(alpha = 0.5, color = "#2E86AB") +

  # Labels for selected points
  geom_text_repel(
    data = label_points,
    aes(label = Name),
    size = 3,
    box.padding = 0.5,
    point.padding = 0.3,
    max.overlaps = Inf
```

```
  ) +

  annotate(
    "text",
    x = 0.05,
    y = min(no_rooks$`pred_K%`, na.rm = TRUE) - 0.02,
    label = "Over-predict",
    vjust = -0.5,
    size = 4
  ) +
  annotate(
    "text",
    x = -0.05,
    y = min(no_rooks$`pred_K%`, na.rm = TRUE) - 0.02,
    label = "Under-predict",
    vjust = -0.5,
    size = 4
  ) +
  annotate(
    "text",
    x = max(no_rooks$pred_diff, na.rm = TRUE),
    y = 0.225,
    label = "League Avg",
    hjust = 1,
    vjust = -0.5,
    size = 4
  ) +

  labs(
    title = "Non-New Pitcher Predictions",
    x = "Prediction Difference",
    y = "Predicted K%"
  ) +
  theme_minimal(base_size = 12)
```



We can see the model slightly under-predicts high K-rate players like Mason Miller and Josh Hader, while is accurate for pitchers like Mason Montgomery and Tyler Alexander. The model under-predicts pitchers like Lucas Gilbreath and Ty Adcock, while over-predicts for low usage pitchers like Joe La Sorsa and Randy Dobnak. However for those pitcher I just mentioned, they have a low sample of batters faced, so it makes sense our model will over/under predict on those pitchers.

```
pct_over <- mean(no_rooks$pred_diff > 0.05, na.rm = TRUE) * 100
pct_under <- mean(no_rooks$pred_diff < -0.05, na.rm = TRUE) * 100
pct_close <- mean(abs(no_rooks$pred_diff) <= 0.05, na.rm = TRUE) * 100

print(paste0("Over-predicts: ", round(pct_over, 1), "%"))
```

```
## [1] "Over-predicts: 17.3%"
```

```
print(paste0("Under-predicts: ", round(pct_under, 1), "%"))
```

```
## [1] "Under-predicts: 12.2%"
```

```
print(paste0("Close/accurate (within ±0.05): ", round(pct_close, 1), "%"))
```

```
## [1] "Close/accurate (within ±0.05): 70.4%"
```

Discussion

So overall our model underperforms. Our model explains ~20% of 2025's pitcher variance, and is 43.7% correlated to the actual K% . Pitchers with fluke seasons and not enough samples get weighted too much. For non-new pitchers our model actually seems to perform decent, however on new or low usage pitchers the model has trouble with. Navigating those roadblocks have been difficult and the model is not as solid as I would have like it to have performed That said, I believe the model doesn't necessarily fail, as it still closely predicts ~70% of non-new pitchers within a 5% threshold of the true K% for the 2025 season. I believe with more work and more data, the model can improve to become a standard predictor of strikeout rates for seasons to come.

Future considerations include a separate model for rookies, refining features, in-depth hyperparameter tuning, and adding more historical data as well as additional features that could play a role in prediciting strikeout rates.